TASK 2 – Explore Values and Types in the Interpreter

Interpreter evidence and explanation

The following session demonstrates the requested values and operators. int represents whole numbers, float represents binary floating-point numbers, str represents text, bool represents True or False, and list is a mutable ordered collection. The slash operator performs true division, // performs floor division, and % returns the remainder. Floor division rounds toward negative infinity, so -7 // 2 is -4.

Interactive session

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
```

Floating-point and money

Most decimal fractions cannot be represented exactly in binary floating point. Therefore 0.1 + 0.2 is stored as a nearby binary value rather than exactly 0.3. Money should not be accumulated as float because tiny representation errors can accumulate over many payments. Integer paise gives exact arithmetic when all values can be represented as whole paise. decimal.Decimal constructed from strings represents decimal amounts exactly and is suitable for financial calculations. round() on floats can reduce displayed decimal places but does not make the underlying binary representation exact.

Complete the Code – TODO 1

Decimal(amount) converts the original decimal text directly to an exact Decimal value. Decimal(float(amount)) first creates an inexact binary float and then converts that approximation, so the small binary error is retained.

Completed program

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3, Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
print("Decimal from float:", sum((Decimal(float(x)) for x in payments), Decimal("0")))
```

Expected output and alternative analysis

The assignment’s expected output is float : 33400.30000000061, Decimal: 33400.30, and False True. Integer-paise alternative: convert each amount to paise (for example, Rs 33.40 becomes 3340), sum integers, and divide by 100 only for display. This is exact if each amount is already rounded to a whole paisa. Decimal addition and integer addition both take O(n) time for n amounts and O(1) extra accumulator space. A float total is also O(n), but is not guaranteed to equal the exact bank total. sys.getsizeof(value) can be used to compare the implementation’s memory occupied by one value; the result is implementation-dependent and excludes referenced objects in containers. A float-round-after-each-addition alternative still relies on binary floating point and can introduce rounding at every step; it is less robust than Decimal or integer paise.

Code file

Q2_program.py